

ЛАБОРАТОРНАЯ РАБОТА № 6

ИНТЕРФЕЙС НА ЕСТЕСТВЕННОМ ЯЗЫКЕ В ЭКСПЕРТНЫХ СИСТЕМАХ

Цель работы.

Целью работы является изучение лингвистических возможностей языка
Пролог.

6.1 Задание на лабораторную работу.

Дополнить реализованную в Лабораторной работе №5 Экспертную Систему пользовательским интерфейсом на Естественном (русском) Языке. Подсистема понимания Естественного Языка реализуется на основе описанной ниже стратегии с использованием ключевых слов. Для формирования пользователем запроса на русском языке в произвольной форме в состав системы должен быть включен текстовый редактор.

6.2 Краткие теоретические сведения.

6.2.1 Проблема понимания компьютером Естественного Языка.

Под Естественным Языком (ЕЯ) в лингвистике понимается любой язык общения между людьми. Под естественностью некоторого языка понимается наличие синонимии и омонимии слов и словосочетаний, а также возможность присутствия свободного порядка слов в предложении.

Целью исследований в области обработки естественного языка является построение программных средств, обеспечивающих восприятие и генерацию ЕЯ-текстов. При этом текст рассматривается как подлежащее восприятию дискретное представление ЕЯ-высказывания. ЕЯ-высказывание здесь представляется в том виде, в котором текстовый материал вводится с клавиатуры. Отдельную область исследований составляет звуковое распознавание и синтез звучащей речи как перевод речевого сигнала в текстовое представление и наоборот.

6.2.2 Виды анализа ЕЯ-информации.

Наиболее известными являются следующие подходы к решению задач ЕЯ-общения конечного пользователя с ЭВМ :

- Прагматический анализ - наиболее сложный, связан с изучением смысла предложения в связи с внеязыковой действительностью. Анализ ключевых слов — метод анализа ЕЯ-высказываний на предмет наличия ключевых слов, которые становятся значениями аргументов предикатов. При этом компьютер одинаково реагирует на различные варианты входного текста, наличие грамматической правильности предложений не является обязательным, роль играет лишь наличие ключевых слов. Применение - построение ЕЯ-интерфейсов к Базам Данных (БД).

- Грамматический анализ: контекстно-свободный и контекстно-зависимый. Контекстно-Свободный (КС) анализ — ЕЯ-фразы классифицируются в зависимости от их внутренней структуры вне зависимости от контекста в соответствии с грамматическими правилами, задающими порядок следования допустимых языком символов (слов). Здесь следует отметить синтаксический анализ предложений.
- Прагматический анализ — наиболее сложный, связан с изучением смысла предложения с учетом связи с внеязыковой действительностью.

6.2.3 ЕЯ-интерфейс к Структурированным Источникам Данных на основе ключевых слов.

Среди известных на сегодняшний день типов Структурированных Источников Данных (СИД) [2] следует выделить реляционные и объектные СУБД, XML и RDF.

Наиболее распространенными типами СИД на сегодняшний день являются реляционные и объектные Базы Данных (БД). XML и RDF относятся к Internet-хранилищам, для которых характерна меньшая степень структуризации, чем для Баз Данных [2].

При разработке ЕЯ-интерфейса к БД на основе ключевых слов с ключевым словом связывается либо характер действия программы (что хочет пользователь), либо значение некоторого атрибута искомого объекта.

Процедура анализа входного текста с целью выделения ключей состоит из трех шагов:

- Ввод команды как строки символов.
- Преобразование введенной строки в список слов.
- Идентификация ключевых слов.

Для преобразования строки в список слов требуется написание правила, которое преобразует строку в список. Для этой цели служит встроенный предикат `fronttoken`. Однако данный предикат предназначен для работы с предложениями на английском языке и не работает со строками, записанными на русском. Здесь программист должен написать свою процедуру разделения русского предложения на слова.

Рассмотрим пример реализации такой процедуры средствами Visual Prolog[^].

```
/* Выделение подстроки до первого разделителя */
fronttoken_cyr(Str, Token, Rest_of_string): —
    symbol_counter(Str, Number), frontstr(Number, Str,
    Token, Rest_of_string).

/* Подсчет символов в строке до конца строки, либо ближайшего пробела, символа
возврата каретки, перевода строки, !, «», #, $ */
symbol_counter(«,», 0).

/* Разделитель в строке — пример для пробела */
symbol_counter(Str, 0): —
    frontchar(Str, '\32', _), !.
```


/* Аналогично описывается условие окончания рекурсии для других разделителей */

```
symbol_counter(Str,Number): —  
    frontchar(Str, _Char, Rest_of_string),  
    symbol_counter(Rest_of_string, Number1),  
    Number=Number1+1.
```

/* Удаление разделителя в начале строки */

```
del_front_space(«»,«»).  
del_front_space(Arg, Res): —  
    frontchar(Arg, Char, Res1), Char='\32', !,  
    del_front_space(Res1, Res).
```

/* Аналогично описывается условие окончания рекурсии для других разделителей */

```
del_front_space(Arg, Arg): — frontchar(Arg, Char, _),  
not(member(Char,['\327\107\137\33\  
'\34','\35','\36','\40','\41',  
'\44','\45','\46','\59'])).
```

/* Предикат upper_lower для кириллицы Windows */

```
upper_lower_cyr(InString, OutString): —  
    str_char_list(InString,Char_List_for_InString),  
    upper_lower_cyr_convers(Char_List_for_InString,  
        Char_List_for_OutString),  
    pack(Char_List_for_OutString,OutString).  
upper_lower_cyr_convers([],[]).  
upper_lower_cyr_convers([Char|Char_List],[Char1|Char_List1]): —  
    char_int(Char,ASCII_code),  
    ASCII_code>=192,ASCII_code<=223,!,  
    ASCII_code_new=ASCII_code+32,  
    char_int(Char1,ASCII_code_new),  
    upper_lower_cyr_convers(Char_List,Char_List1).  
upper_lower_cyr_convers([Char|Char_List],[Char1|Char_List1]): —  
    char_int(Char,ASCII_code),  
    ASCII_code=168,!,  
    ASCII_code_new=ASCII_code+16,  
    char_int(Char1,ASCII_code_new),  
    upper_lower_cyr_convers(Char_List,Char_List1).  
upper_lower_cyr_convers([Char|Char_List],[Char1|Char_List1]): —  
    char_int(Char,ASCII_code),  
    ASCII_code>=65,ASCII_code<=90, !,  
    ASCII_code_new=ASCII_code+32,  
    char_int(Char1,ASCII_code_new),
```



```

upper_lower_cyr_convers(Char_List,Char_List1).
upper_lower_cyr_convers([Char|Char_List],[Char|Char_List1]): —
    upper_lower_cyr_convers(Char_List, Char_List1).
/* Преобразование строки в список символов */
str_char_list(«,[]).
str_char_list(Word,[Char|Char_List]): — frontchar(Word,Char,WordRest),
    str_char_list(WordRest,Char_List).
/* Превращение списка символов в строку */
pack([ ],«»).
pack([H|T], Res): — str_char(Str_H, H), pack(T, Res1), concat(Str_H,
    Res1, Res).
/* Модифицированное правило преобразования строки в список слов */
convers(«,[]): — !.
convers(Str, [Head1|Tail]): — fronttoken_cyr(Str, Head, Str2),
    upper_lower_cyr(Head, Head1), del_front_space(Str2, Str1),
    convers(Str1, Tail).

```

Идентификация ключевых слов может быть реализована двумя способами. Первый состоит в задании всех ключевых слов в утверждениях БД. Тогда внутренние унификационные процедуры Пролога смогут сопоставить элементы предложения со значениями, взятыми из БД. Второй способ основан на частоте применения определенных грамматических конструкций : ключевые слова определяются позицией, которую они занимают в предложении. В примере для реализуемой нами Экспертной Системы в качестве ключевых берутся первое (Kname) и последнее (Lname) слово ЕЯ-высказывания. При этом первое слово ассоциируется с выполняемым программой действием, второе — с объектом из БД, над которым производится действие.

Пример для варианта Экспертной Системы, базирующегося на логике : /*

```

Правило для проверки ключевых слов */
do_right_form(Kname, Lname): — func_keyword(Kname),
    rule(_,Lname,_,_), !.
do_right_form(Kname,_): — frontstr(3, Kname, Word, _),
    upper_lower_cyr(Word, Key), Key=«вых»,!, exit.
do_right_form(Kname,_): —
    upper_lower_cyr(Kname, Word),
    Word=«exit», !,
    exit.
do_right_form(Kname,_): —
    upper_lower_cyr(Kname, Word),
    Word=«quit», !,
    exit.

```



```

do_right_form(Kname, Lname): —
    concat(«Введенные Вами ключевые слова : », Kname, Temp),
    concat(Temp, « и », Temp1), concat(Temp1, Lname, Temp2),
    concat(Temp2, « не известны системе. », Msg), write(Msg), !.
/* Пример проверки допустимости ключевого слова-приказа */
func_keyword(Word): — frontstr(3, Word, Key, _),
    upper_lower_cyr(Key, KeyTr), KeyTr=«най», !.

```

6.3 Создание окна текстового редактора в Visual Prolog.

В Visual Prolog для создания текстовых редакторов эксперт окон и диалоговых окон в составе среды визуальной разработки имеет особый стиль программного кода, который генерируется экспертом кода для окна редактора, стиль `edit_Create`. Для настройки и использования редактора существует гибкий API (Application Programming Interface) [1, с. 680].

При настройке созданного редактора рекомендуется в сгенерированном по умолчанию коде окна редактора указать задаваемую предикатом `font_Create` семейство и размер шрифта для редактируемого текста (в частности, с учетом наличия поддержки кириллицы). Кроме того, в генерируемом по умолчанию коде предиката создания окна редактора отсутствует указание источника редактируемого текста. Редактируемый текст можно сделать одним из аргументов указанного предиката. Пример :

```

win_грамматический_разбор_Create(_Parent, Text): —
ifdef use_editor
    Font = font_Create(ff_System, [ ], 10), ReadOnly = b_false, Indent =
        b_true, InitPos = 1,
        edit_Create(win_грамматический_разбор_WinType,
        win_грамматический_разбор_RCT,
        win_грамматический_разбор_Title,
        win_грамматический_разбор_Menu, _Parent,
        win_грамматический_разбор_Flags, Font, ReadOnly,
        Indent, Text, InitPos, win_грамматический_разбор_ch),
endif
    true.

```

6.4 Содержание отчета по работе.

- 1) Формулировку цели и задач проводимых исследований;
- 2) Анализ задания, выбор метода решения с обоснованием, описание процесса разработки программ, полученные результаты и их анализ (обоснование);
- 3) Тексты программ с комментариями и обоснованиями;
- 4) Выводы по проведенным машинным экспериментам.